UNIT 2 60 min

Data Types and Strings

Integers, floats, strings, and booleans; checking types; converting between them; working with text.

By the end of this unit you will be able to

- Identify the common types: `int`, `float`, `str`, `bool`
- Check a value's type with `type()` and convert with `int()`, `float()`, `str()`
- Explain why `0.1 + 0.2` is not exactly `0.3`
- Index and slice strings, and use common string methods
- Build readable output with f-strings

Every value has a type

A **data type** tells Python what kind of value something is and what operations make sense on it. Python checks types as the program runs.

Type	Name in code	Examples	What it holds
Integer	`int`	`42`, `-7`, `0`	Whole numbers
Float	`float`	`3.14`, `-0.5`, `2.0`	Numbers with a decimal point
String	`str`	`"Hello"`, `'a'`	Text, in quotes
Boolean	`bool`	`True`, `False`	Yes or no. Capitalized.

types.py

```
age = 15          # int
height = 5.8      # float
name = "Jordan"   # str
is_student = True # bool

print(type(age))
print(type(height))
print(type(name))
print(type(is_student))
```

Arithmetic and division

Python has two division operators. `/` always returns a float. `//` is floor division and returns a whole number. `%` is the remainder, which is useful for checking even and odd.

division.py

```
print(7 / 2)    # 3.5
print(7 // 2)   # 3
print(7 % 2)    # 1
print(2 ** 10)  # 1024
```

Converting between types

Input and file data often arrive as text. Convert with `int()`, `float()`, or `str()`. This is called **casting**.

casting.py

```
text = "42"
number = int(text)
print(number + 8)     # 50 (math)
print(text + "8")     # 428 (joins two strings)
print(float("3.5") * 2)
print(str(99) + " bottles")
```

Watch out

`int("3.7")` raises a `ValueError`. Convert to `float` first, then to `int` if you need to drop the decimal part: `int(float("3.7"))` gives `3`.

Floats are approximate

Computers store most decimal numbers in binary, which cannot represent 0.1 exactly. Small errors show up. Compare floats with a tolerance, not with `==`. Later you will use `math.isclose()`.

float_surprise.py

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(round(0.1 + 0.2, 2))
```

Strings

A string is an ordered sequence of characters. Each character has an **index**, starting at 0. Negative indexes count from the end. A **slice** `text[start:stop]` returns a part of the string; `stop` is not included.

strings.py

```
word = "Python"
print(word[0])      # P
print(word[-1])     # n
print(word[0:3])    # Pyt
print(word[2:])     # thon
print(len(word))    # 6
print(word.upper()) # PYTHON
print("py" in word.lower())  # True
```

Strings are **immutable**. You cannot change one character in place. You build a new string instead.

> **Watch out**
> `word[0] = "J"` raises `TypeError: 'str' object does not support item assignment`. Use `"J" + word[1:]` to build `"Jython"`.

Method	What it does	Example result
`.upper()` / `.lower()`	Changes letter case	`"hi".upper()` → `"HI"`
`.strip()`	Removes spaces at both ends	`"  hi  ".strip()` → `"hi"`
`.replace(old, new)`	Replaces every match	`"a-b".replace("-", "+")` → `"a+b"`
`.split(sep)`	Splits into a list	`"a,b".split(",")` → `["a", "b"]`
`.startswith(text)`	Checks the beginning	`"Python".startswith("Py")` → `True`

f-strings

Put an `f` before the opening quote and wrap any expression in braces. This is the cleanest way to mix text and values.

fstrings.py

```
score = 92
student = "Maya"
print(f"{student} scored {score}% on the test.")
print(f"Next goal: {score + 3}%")
```

Predict the output THINK FIRST

Work out each result by hand first.

```
print(7 / 2)
```

Answer: `3.5`

```
print(7 // 2)
```

Answer: `3`

```
print("5" * 3)
```

Answer: `555`
Multiplying a string repeats it.

```
print(bool(""))
```

Answer: `False`
An empty string is "falsy".

```
print(int("3.7"))
```

Answer: `ValueError`
A decimal string cannot be parsed as an int.

Practice

▼ **Exercise 1 — Full name**

Store a first name and a last name in two variables. Print the full name using an f-string, for example `Ada Lovelace`.

▼ **Show solution**

Solution

```
first = "Ada"
last = "Lovelace"
print(f"{first} {last}")
```

▼ Exercise 2 — Clean and count

Start with `raw = "  Python Is Fun  "`. Remove the extra spaces, make it lowercase, and print how many characters it has.

▼ **Hint**

Methods can be chained: `raw.strip().lower()`. Then call `len()` on the result.

▼ **Show solution**

Solution

```
raw = "   Python Is Fun   "
clean = raw.strip().lower()
print(clean)
print(len(clean))
```

Discussion

1. Why do you think `"5" + "5"` gives `"55"` while `5 + 5` gives `10`? What does that tell you about types?
2. Where in your daily life do you see numbers stored as text, and what would break if a program treated them as numbers?
3. If floats cannot represent 0.1 exactly, when should a bank program use integers (cents) instead?

▼ **Facilitator notes**

The bank question is a good opening for a short discussion of money and `Decimal`. Students do not need to implement it yet.

Key terms

Data type

A category of value (int, float, str, bool) that determines what operations are valid on it.

int

Integer: a whole number with no decimal point.

float

Floating-point number: a number with a decimal point. Stored approximately.

str

String: an immutable sequence of characters written in quotes.

bool

Boolean: the value `True` or `False`. Named for George Boole.

type()

A built-in function that returns the type of a value, such as `<class 'int'>`.

Casting

Converting a value from one type to another, for example `int("42")`.

Index

The position of an item in a sequence. Counting starts at 0.

Slice

A part of a sequence written `seq[start:stop]`. The stop position is excluded.

Immutable

A value that cannot be changed after it is created. Strings, integers, and tuples are immutable.

f-string

A string literal beginning with `f` that embeds expressions in braces, such as `f"{x}"`.

Method

A function that belongs to an object and is called with a dot, such as `name.upper()`.

See all terms in the glossary →